TECHNICAL REPORT

Task 5: Documentation and Demonstration

Project Title

Blockchain-Based Academic Certificate Verification System

Technology: Python Flask, HTML, CSS, JavaScript

1. Introduction

The Blockchain-Based Academic Certificate Verification System is a web-based application designed to verify academic certificate information. The system provides user authentication, manual certificate verification, PDF certificate processing and an AI Assistant for application guidance.

2. System Architecture

The application follows a Flask-based web architecture. The user interacts with the frontend through HTML, CSS and JavaScript. Flask handles routing and backend processing. Student records are read from an Excel database. PDF certificates are processed using PyPDF2 and scanned certificates can be processed using Tesseract OCR. The AI Assistant communicates with the OpenAI API through the /api/chat endpoint.

Layer	Component	Function
Frontend	HTML, CSS, JavaScript	User interface and interaction
Backend	Python Flask	Routes and application logic
Database	students.xlsx	Student and Certificate ID records
Document Processing	PyPDF2 / Tesseract OCR	Extract certificate text
AI Service	OpenAI API	AI Assistant responses

3. API Documentation

Route	Method	Purpose
/	GET	Redirects user to registration
/register	GET / POST	User registration
/login	GET / POST	User authentication
/dashboard	GET / POST	Certificate verification and dashboard
/api/chat	POST	AI Assistant communication
/logout	GET	Clears session and logs out user

4. Certificate Verification Flow

The user first logs into the application and opens the Check Certificate section. The user can either enter the student name and Certificate ID manually or upload a PDF certificate. For manual verification, the entered details are compared with the Excel student database. For PDF verification, text is extracted using PyPDF2. If the PDF does not contain extractable text, Tesseract OCR is used. The extracted student name is matched with the registered records. The result is then displayed as Verified, Failed or Error.

5. Sequence of Application Operation

Step	Actor / Component	Action
1	User	Opens the application
2	Flask	Displays registration page

Step	Actor / Component	Action
3	User	Registers and logs in
4	Dashboard	Displays verification options
5	User	Enters details or uploads certificate
6	Backend	Processes and verifies certificate
7	Excel Database	Matches student records
8	System	Displays verification result

6. Source Code Repository

The project source code is maintained in a GitHub repository. The repository contains the Flask backend, HTML templates, CSS and JavaScript files, project data and required project documentation.

Repository Link: To be added after uploading the project to GitHub.

7. Demonstration Plan

The project demonstration will show the complete working flow of the application. The demonstration starts with user registration and login. The dashboard is then opened and the manual certificate verification feature is tested. PDF certificate upload and automated text/OCR processing are demonstrated. Finally, the AI Assistant is used to answer a question related to the application.

8. Conclusion

The technical documentation describes the architecture, API routes and certificate verification flow of the Flask application. The project is supported by a source code repository and can be demonstrated through its major modules. The documentation and demonstration activities provide a complete overview of the developed academic certificate verification system.